

WHITEPAPER · CREDITCOIN CC3 × ETHEREUM · USC

TruthGate

An On-Chain Credit Bureau on the Attestcoin Protocol

Every credit record is a cryptographic proof.

Version 1.0 · September 12, 2026

Repository — github.com/henessay/truthgate

Live dashboard — truthgate-six.vercel.app

Demo video — youtu.be/Vayw9FCrc1U

1 · The problem and the market

Roughly two billion adults are unbanked or credit-invisible: they may save, borrow informally, and repay reliably, yet no institution holds a record that any lender will trust. A credit bureau is, at its core, a registry of financial facts about a person — and for most of the world that registry either does not exist or cannot be independently verified.

Public blockchains looked like the answer: every deposit, repayment, and liquidation is already recorded on a ledger nobody can rewrite. In practice, on-chain reputation has remained unverifiable at the point of use. A lending protocol on chain A cannot read chain B; so every "cross-chain reputation" system to date has inserted a trusted party in the middle — an indexer whose database is taken at its word, an oracle committee, or the project's own backend signing attestations about what it saw. The record may be public, but the *reading* of it is trusted, which quietly reduces the bureau to the credibility of whoever operates the middleware.

Prior attempts also stumbled on a subtler problem than transport: **self-attestation**. If any contract can emit "user X repaid me," then proving the event is genuine does not make the fact meaningful — anyone can deploy a contract and write themselves a spotless history. A credit bureau needs both properties at once: records that are *cryptographically verifiable* end-to-end, and a structural answer to *who is allowed to write history and what it costs to lie*. TruthGate is built as exactly that pair: an Attestcoin-proof-fed registry of financial facts (nothing enters without a validity proof verified on-chain), and a tiered source-legitimacy model that prices the right to write (§4).

2 · Architecture

2.1 Three layers

1. **Proven-facts registry.** Every fact enters through one pipeline (`TruthGateBase.execute`): `chainKey` pin → freshness window → `queryId` anti-replay → precompile `verifyAndEmit` → receipt-status check → log parsing. Emitters carry an on-chain trust tier — the structural fix for the self-attestation problem: *a proof shows contract Y emitted an event, not that Y is an honest lender.*
2. **Reference scoring.** CAPITAL (amount-weighted, joint cap across protocols) + DISCIPLINE (flat per proven repayment, bond-capped, dark until the capital gate passes) + NEGATIVE (flat per proven liquidation, degrades but never hard-blocks) + local CC3 repayment history (risk-proportional). No cross-asset arithmetic without oracles — heterogeneous amounts are emit-only, by rule.
3. **Demo credit line.** A CTC loan book (LPPool) that prices limits off the bureau, with two repayment paths: native CTC on CC3, or USDC locked on Ethereum and delivered by proof (wUSDC mint + treasury swap). Deliberately thin — the first consumer of the data, not the product.

Remove the proofs and the product ceases to exist. TruthGate has **no other data source**: no oracle feed, no off-chain indexer whose word is trusted, no admin method that writes a score. Every unit of every score component traces to a `verifyAndEmit` call on the CC3 block-prover precompile:

- **CAPITAL** (`ethScore`) — grows only from proven `FundsDeposited` / staking events on Sepolia.
- **DISCIPLINE** (`disciplineScore`) — grows only from proven repayment events, including a real Morpho Blue Sepolia `Repay` delivered on-chain.
- **NEGATIVE** (liquidation penalty) — applied only from proven `LiquidationCall` events; a third party can permissionlessly prove a borrower's liquidation.
- **Path B repayments** — wUSDC is minted on CC3 only against a proven `UsdcLockedForRepayment` on Sepolia; the bridge holds no other mint path.

2.2 The proof loop (Sepolia event → CC3 state change)

1. A financial event fires on Sepolia — e.g. `Repay` on the official Morpho Blue singleton.
2. Attestcoin attests the Sepolia block on CC3. Measured: **~470 s (~8 min)** for a fresh block — the lag is Attestcoin's design (attestation deliberately trails the source head as reorg protection); **~6 s** if the block is already attested.
3. The off-chain worker builds the proof via the Prover API (`ProofBuilder.getProof(txHash)` from `@gluwa/usc-sdk`) → {`txBytes`, `MerkleProof`, `ContinuityProof`}. Measured: **~0.4–0.7 s** for an attested block (prover cache).
4. Optional free pre-check: the same proof through `verify` as an `eth_call` (`staticCall`) — the worker spends zero gas on proofs that would revert. This fired live: an over-cap repayment was rejected at `estimateGas`, no on-chain transaction was ever sent.
5. The worker submits `execute(chainKey, height, txBytes, merkleProof, continuityProof)` to the consumer contract (CreditCore or RepaymentBridge), which calls the precompile verifier `0x...0FD2 : verifyAndEmit(...)`. Measured end-to-end (verify + decode + business logic): **~10–12 s, ~670k gas** for a scoring delivery, **~1.0M gas** for a Path B repayment (mint + accounting).

6. The precompile proves *inclusion in a finalized Sepolia block* — nothing more. Everything after that is our contract: receipt-status check, log extraction, registry tier weighting, scoring.

2.3 The base contract (`TruthGateBase`)

Every proof enters through one audited pipeline, in this order:

1. **chainKey pin** — `require(chainKey == 1)` (Sepolia's attested-chain key on CC3).
2. **Freshness window** — `height ≥ minAcceptedHeight` where scoring semantics require it (stale-event pushing).
3. **Anti-replay** — `queryId = keccak256(chainKey || height || txIndex)`; `processedQueries[queryId]` is required-unset *before* any processing.
4. **Verification** — `VERIFIER.verifyAndEmit(...)`; the precompile reverts on an invalid proof.
5. **Receipt status** — the precompile does **not** check whether the source transaction succeeded, only that it is included. `EvmV1Decoder.decodeReceiptFields` → `require(receiptStatus == 1)` is our check, mandatory before any business logic.
6. **Log extraction** — filtered by event signature (topic0). Two variants: `_validateAndExtractLogsAnySource` (CreditCore — the emitter's trust level is decided by the registry, per log) and the pinned-emitter `_validateAndExtractLogs` (RepaymentBridge — money movement accepts exactly one registered vault).
7. **Atomicity** — no `try/catch`, no unchecked low-level calls anywhere in `execute` → `handlers`: any failure reverts the whole transaction including the `processedQueries` mark. Guarantee: *queryId marked ⇔ all effects applied* — otherwise a partially-executed proof with a burned queryId would be an unrecoverable hole. Locked by `test_invariant_queryIdMarkedIffEffectsApplied`.

3 · Source legitimacy: who writes your history

Attestcoin's proof answers exactly one question: *was this event emitted by contract Y in a finalized source-chain block?* It cannot answer whether Y is an honest lender. Without a second layer, anyone could deploy a contract that emits `Repay` events and write themselves a credit history — the self-attestation problem, and the central design problem of a proof-fed bureau. CreditCore v4 closes it with three structural fixes:

1. Tier field. Every event emitter carries an on-chain trust tier in the source registry (`sources` mapping):

- **VERIFIED** — canonical protocol deployments pinned against mainnet (Aave, Morpho, Spark, Comet, Rocket Pool, EigenLayer) plus the ScoringVault (its deposit events are intrinsically backed by `msg.value` — it is the capital anchor, and gating capital by bonds would make the capital gate circular). Full weight. Registration is owner-only.
- **BONDED** — any lender contract may join **permissionlessly** via `registerBondedSource()` payable, staking a CTC bond (≥ 1 CTC) behind its records. Weight is capped by the bond (fix 2). Our own LoanBookSim runs at this tier with a real 10 CTC bond posted on-chain — the simulator is no longer silently trusted.
- **UNKNOWN** — everything else (the mapping default). Events still verify and emit (visible history), but contribute **zero** to every score. Negative signals are tier-weighted too: an unregistered contract cannot degrade anyone's score (griefing shield).

2. Per-source bond-cap. For a BONDED source, the total credit-limit influence attributable to its records — positive score deltas converted through the limit slope, and liquidation penalties as-is — is clamped at its staked bond (`attributedLimit` $\leq$ `bond`, linear, constant ratio = the existing `SCORE_SLOPE`). The invariant is literal: *a fake lender cannot write more reputation than it has capital at risk* — attack cost $\geq$ extracted benefit — and symmetrically cannot inflict more limit damage than its bond. Attribution history is never reset: withdrawing and re-posting a bond does not launder it. Bond withdrawal is owner-arbitrated in v4; slashing/challenge of dishonest bonded sources is roadmap, stated as such.

3. Capital gate. DISCIPLINE contributes to a borrower's limit only while that borrower has independently proven CAPITAL of at least `CAPITAL_GATE_THRESHOLD` (0.015 ETH) — an indicator multiplier evaluated at read time, retroactive in both directions. A self-made lender cannot farm score for a wallet that has no real skin in the game. The threshold is a demo-scale constant; production pools calibrate to their loan sizes.

Dust-repay floor and the discipline cap. External repayments earn a flat `EXTERNAL_REPAY_FLAT_SCORE` (0.1) per proven event — amounts are heterogeneous loan-token units and stay emit-only (§5). Flat credits invite dust spam (1-wei repays on a permissionless market), so two bounds apply: a per-event floor `minDisciplineEventAmount` (default $1e4$ raw loan-token units — a deliberately decimals-unaware heuristic; it kills literal-wei spam, nothing more) and the per-borrower `DISCIPLINE_SCORE_CAP` (1.0 = at most +10 CTC of limit from discipline) as the economic bound. Stated honestly: an attacker who passes the capital gate can still farm gas-price repays on a verified permissionless protocol **up to the cap** — bounded, not eliminated; elimination needs oracle-weighted amounts (roadmap).

3.1 Reputation earned on CC3 itself: size $\times$ time, not repayment count

localScore grows for repaid CreditCore loans — proportionally to the risk the pool actually took on, not to the mere fact of repayment:

```
localScoreDelta = principal × min(heldBlocks, LOAN_DURATION_BLOCKS) / LOAN_DURATION_BLOCKS  
/ LOCAL_SCORE_NORM_PRINCIPAL
```

- `heldBlocks` — CC3 blocks from issuance to the closing payment; `MIN_HOLD_BLOCKS = LOAN_DURATION_BLOCKS/4` — repayment before the threshold goes through normally but accrues no score. A late repayment rehabilitates (clears debt, unblocks borrowing) but accrues nothing.
- Why the "borrow-and-repay cycle" attack fails: an instant cycle yields exactly 0 (hold threshold + zero-time proportion); above the threshold, gaining +1 CTC of limit requires holding ~4.5 CTC for the full term and paying ~0.225 CTC of interest — the limit costs $\geq \sim 22.5\%$ of its value in paid interest, however it is split across loans. The only way to farm localScore is to actually use credit and pay for it — exactly the behavior being measured.
- The time parameters are constructor-set (immutable); the demo deploy compresses them (~240-block term vs. the hardcoded production default of 100,000) so the "reputation → more credit" flywheel is observable live. The reward depends on the **fraction** of the term held, so proportional compression preserves the economics (locked by `test_scoreScaleInvariantUnderCompressedSchedule`).

CAPITAL is counted once, not once per protocol. Deposits are a weak signal — the same funds can be cycled deposit→withdraw→deposit, or restaked across protocols — so all CAPITAL sources draw from one shared accumulator against a single `DEPOSIT_SCORE_CAP` (2 ETH) per address (`test_jointCapitalCap_capitalCountedOnce`). **A proven liquidation degrades but never hard-blocks:** the penalty burns only the earned bonus — the base limit always survives, and `ethScore` is untouched, so negative proofs cannot be weaponized into a lockout.

4 · The boundary of provability

Attestcoin proves one class of statement: *this transaction, with this receipt and these logs, is included in a finalized source-chain block*. Three consequences shape the bureau's design; each is documented rather than papered over.

4.1 State-dependent event meaning: why Comet DISCIPLINE is unprovable

Compound v3 has **no repayment event**. Repaying debt is `supply()` of the base asset into an account whose principal is negative; the contract emits the same `Supply(address indexed from, address indexed dst, uint256 amount)` whether the caller is a lender depositing or a borrower repaying. Which one it was is determined by the **sign of the account's principal before the transaction** — contract *state*, not transaction content.

USC proves a transaction's inclusion and with it the receipt — its events. It does not (and cannot, in this model) attest arbitrary historical contract state at that block. So a proven `Supply` cannot be trustlessly classified as a repayment: doing so would require trusting an archive-node lookup or an oracle for the pre-transaction balance sign — exactly the trust assumption the bureau exists to avoid. The failure mode of guessing would be score farming: any lender deposit would mint DISCIPLINE for free. `AbsorbDebt` has no such ambiguity — it fires only when the protocol absorbs an underwater account — so Comet contributes NEGATIVE only. This is a semantic property of Comet's event design, not a parser gap: it is documented as unprovable rather than left as silent non-coverage.

4.2 Existence, never absence: wallet age is only provable as a lower bound

An inclusion proof is an existential statement. It can establish "*this wallet had already transacted at height H*" — which lower-bounds the wallet's age — but it can never establish "*this was the wallet's first transaction*" or "*this wallet has never been liquidated*": those are universally-quantified claims over **all** source-chain history, and no finite set of inclusion proofs expresses them.

The bureau's consequence is structural: it scores the **presence of proven good facts** and the **presence of proven bad facts**, never a certified *absence* of bad ones. A borrower can prove they deposited, repaid, or recovered after a liquidation; nobody can prove a clean history — and the design does not pretend otherwise. The honest complement is that negative facts are third-party-provable: anyone can deliver a proof of your liquidation (tier-weighted, so only registered sources count). A production "wallet age" signal would follow the same rule: credit the proven lower bound ($\text{age} \geq \text{now} - H$ of the earliest proven transaction), never a claimed exact age.

4.3 No cross-asset arithmetic without oracles

Event amounts across protocols are denominated in heterogeneous tokens: 500 USDC (6 decimals) and 500 DAI (18 decimals) differ by $1e12$ as raw values while representing the same economics; Morpho amounts are per-market loan-token units; EigenLayer shares are per-strategy LST units. Comparing or summing them trustlessly requires price oracles — a trust dependency the v1 bureau refuses. The design rule, applied uniformly: **heterogeneous amounts are parsed and emitted for transparency but never enter scoring arithmetic**. Liquidation penalties are flat per-event; external repays earn a flat 0.1; EigenLayer deposits credit flat. The two exceptions are principled: native-ETH amounts (vault deposits, Rocket Pool — one homogeneous unit) are amount-weighted under the joint cap, and the dust floor is an

explicitly decimals-unaware heuristic that kills literal-wei spam and claims nothing more. Oracle-weighted amounts are roadmap, and would relax exactly this rule, consciously.

5 · Coverage and live evidence

5.1 Protocol coverage (≈ \$40.2B TVL)

The bureau's parser library extends past the contracts wired into the live Sepolia pipeline. `docs/protocol-registry.json` is the machine-readable registry; `contracts/test/MainnetParity.t.sol` pins every parser against **real historical transactions of the verified live mainnet contracts** — raw log bytes decoded through our parsers with every field asserted, source tx hash cited. A topico match alone cannot catch data-layout drift when a fork keeps the signature string but changes field assumptions; decoding real bytes can. Per-protocol TVL figures: DefiLlama, 2026-08-31.

PROTOCOL	EVENTS	CATEGORY	STATUS	TVL
Aave v3	Repay, LiquidationCall	DISCIPLINE, NEGATIVE	mainnet-pinned + Sepolia live (action live, watcher armed)	\$17.2B
Spark (SparkLend)	Repay, LiquidationCall	DISCIPLINE, NEGATIVE	mainnet-pinned (byte-identical Aave fork; pool identity checked on-chain — same parser, no code of its own)	\$4.4B
Morpho Blue	Repay, Liquidate	DISCIPLINE, NEGATIVE	mainnet singleton pinned + Sepolia live pipeline — real Repay delivered on-chain	\$9.5B
Compound v3 (Comet)	AbsorbDebt	NEGATIVE only	mainnet-pinned; DISCIPLINE provably unprovable (\$4.1)	\$1.4B
Rocket Pool	DepositReceived	CAPITAL	mainnet-pinned (address resolved live from RocketStorage — upgradeable contracts)	\$1.3B
EigenLayer	Deposit	CAPITAL (flat; joint cap)	mainnet-pinned, current slashing-era layout (the remembered older layout produced zero logs on the live proxy and is deliberately unsupported)	\$6.4B
Maker/Sky · Euler v2 · Fluid · Curve	—	—	roadmap (Maker's anonymous-event model needs a calldata adapter, not a log parser)	—

5.2 Consumed events (live Sepolia → CC3 pipeline)

EVENT	SOURCE & TIER	EFFECT
FundsDeposited(depositor, amount, nonce)	ScoringVault — Verified	ethScore += amount, joint CAPITAL cap 2 ETH
LoanRepaidOnEth(borrower, loanId, amount)	LoanBookSim — Bonded, 10 CTC	disciplineScore += amount + flat bonus, bond-capped
LiquidationCall(...) — Aave v3's real layout, byte-for-byte	LoanBookSim — Bonded, 10 CTC	flat penalty 1 / 2 CTC, cap 5; burns earned bonus only
Repay(reserve, user, repayer, amount, useATokens)	Aave v3 Sepolia Pool — Verified	disciplineScore += 0.1 flat (watcher armed)
Repay(id, caller, onBehalf, assets, shares)	Morpho Blue Sepolia singleton — Verified	disciplineScore += 0.1 flat — delivered live
UsdcLockedForRepayment(borrower, cLoanId, amount)	RepaymentVault — pinned emitter	wUSDC mint + repayment accounting (≤30% of the debt)

5.3 v4 go-live evidence (2026-09-01)

All nine historical Sepolia proofs replayed with fresh proof deliveries, plus the real Morpho Repay as the tenth — zero failures:

#	REPLAYED SEPOLIA EVENT	CC3 DELIVERY TX	RESULT
1–6	six ScoringVault deposits	0xdc6ae575...15e3 · 0x350e4505...a64f · 0xe4f38752...dcd8 · 0xcc66cb58...fa4c · 0x5a256def...ae12 · 0xd0e44cd4...c0d8	EthScoreIncreased ×6, capital 0.027 restored
7	LoanBookSim repayment sim	0x0e26ee28...194c	DisciplineScoreIncreased +0.35 (3.5 CTC attributed against the 10 CTC bond)
8–9	two LoanBookSim liquidations	0x1bc80bdd...85ab · 0x0c8374fd...6133	LiquidationPenaltyApplied 1 + 2 CTC (attribution 6.5/10 total)
10	Morpho Blue Sepolia Repay 0xf232bbd7...ee17	0x5d32a354...af38b	DisciplineScoreIncreased +0.1 — the first real external-protocol DISCIPLINE record (50e6 assets clear the dust floor by 5000×)

5.4 Post-replay bureau state (reconciled against the arithmetic)

QUANTITY	VALUE	RECONCILIATION
capital (ethScore / depositScoreOf)	0.027	six proven deposits, joint cap untouched ✓
disciplineScore	0.45	0.35 (sim repay) + 0.1 (Morpho flat) ✓
capital gate (threshold 0.015)	PASSED at 1.8×	0.027 ≥ 0.015 → discipline counts ✓
effectiveScore	0.477	0.027 + 0.45 ✓
liquidationPenalty	3.0	1 (first) + 2 (repeat) ✓
creditLimit	6.67 CTC	5 (BASE) + (0.477 – 0.01)×10 – 3 ✓
sources[LoanBookSim]	Bonded 10, attributed 6.5	3.5 (discipline×slope) + 3.0 (penalties) ≤ bond ✓

Snapshot at v4 go-live; the live state evolves as the demo borrower keeps using credit — current values on the Overview screen of the live dashboard.

5.5 Protections that fired on live networks

- **Anti-replay by queryId:** re-delivery of an already-processed lock — revert `Query already processed`.
- **Path B 30% cap:** an over-cap repayment rejected with `USDC share cap exceeded` — twice over: the contract require, and the worker spent no gas (revert caught at `estimateGas`; no on-chain transaction sent).
- **Height-scale bug caught by the live run** (fixed): a CC3-block deadline was once compared against a Sepolia height — every repayment reverted; the bridge was redeployed checking against `CC3 block.number`, and the same proof then went through. Kept in the record deliberately: the live run is a test, not a demo reel.

- **CC3 code-size limit:** the v4 core exceeded the 24,576-byte runtime limit; the solc optimizer (200 runs) brought it to 17,928 bytes with the full 88-test suite passing identically.

6 · Trust assumptions and the decentralization roadmap

We enumerate our own trust assumptions. The contract owner can, today: register VERIFIED sources (curation of canonical deployments), arbitrate bonded-source bond withdrawal (no slashing/challenge mechanism yet), tune the dust floor `setMinDisciplineEventAmount`, set the freshness window `setMinAcceptedHeight`, and mark loans expired. Each is a v4 pragmatism, not a design endpoint: the roadmap replaces curation with on-chain verification proofs where possible, bond arbitration with a challenge game, and the knobs with governance.

Everything else is trust-minimized by construction: scores have no admin setter, the bridge has no mint path without a proof, and the AI agent's authority is limited to submit/defer/escalate decisions on proofs — it never holds keys to money.

6.1 Roadmap

- **Bonded-source challenge mechanism (optimistic, UMA-pattern).** Replace owner-arbitrated bond withdrawal with an optimistic challenge game: a withdrawal request opens a dispute window during which anyone may challenge a source's record history by staking against it; unchallenged withdrawals settle automatically, disputed ones escalate to a resolution layer. The bond becomes slashable evidence collateral rather than an owner-guarded deposit.
- **Reverse writability (CC3 → Ethereum), when the protocol ships it.** v1 is read-only by design — locked USDC stays locked, and the economics closes on the CC3 side through the SwapDesk. Attestcoin writability would let the bureau export attestations (e.g., a proven score snapshot consumable by Ethereum lending pools) and let the vault release funds against CC3-side proofs.
- **Identity sources: BABT and ENS.** Sybil-resistance signals that are themselves on-chain events — Binance Account Bound Token holdings, ENS registrations and their age — consumed through the same proof pipeline and the same tier registry, credited under the same existence-only rule (§4.2): presence of a proven identity fact, never certified uniqueness.
- **Per-horizon pools.** Multiple LPPools with different loan durations and risk appetites pricing off one shared bureau — the registry is the product; pools are consumers, each calibrating the capital-gate threshold and limit slope to its own loan sizes.
- **Oracle-weighted amounts.** The single conscious relaxation of the no-cross-asset rule (§4.3): once a price-oracle dependency is accepted, heterogeneous repay/liquidation amounts can enter scoring arithmetic value-weighted instead of flat.

6.2 v1 limitations, stated

- **Identity model: a single EOA on both chains.** A participant's Sepolia address must match their CC3 address — the borrower is identified by the address in the proven event's indexed topic. Smart-contract wallets and differing cross-chain addresses are unsupported; on a mismatch the bridge reverts.
- Loan deadlines are in CC3 blocks, checked against CC3 `block.number` at proof delivery time (+buffer). Sepolia heights take no part in deadline checks — the scales are incomparable.
- The wUSDC/CTC rate is a constant 1:1 (production: an oracle).

7 · Measured performance and references

7.1 Measured performance (live networks)

METRIC	VALUE
Attestation of a fresh Sepolia block on CC3	~470 s (~8 min; the lag is Attestcoin's design – reorg protection)
Wait for an already-attested block	~6 s
Proof generation (Prover API, attested block)	~0.4–0.7 s (cache)
Verification + execute on CC3, end-to-end	~10–12 s
Gas: scoring delivery (execute → CreditCore)	~670k (669,340 / 669,788)
Gas: repayment delivery (execute → RepaymentBridge, mint + accounting)	~1.0M (1,008,350 / 986,174)
Gas: treasury swap <code>swapWusdcForCtc</code>	401,282

7.2 Current deployment

CONTRACT	CHAIN	ADDRESS
CreditCore v4 (the bureau)	CC3	0x02501978888A9BedfED93F9665c7DDeA466f12F5
RepaymentBridge v5	CC3	0x1e9E90d0a47Cc3E080B37F986e1Bf7E946446713
WrappedUSDC	CC3	0xF3663288a86BeeAD53Bf9C275E663bA9c247A7AC
LPPool	CC3	0x1Cc00628a8590e4eFDA496242439d5d09e726C14
ScoringVault	Sepolia	0xe5f1cb738ba279440565D7B46fFAB38a0E52AF37
LoanBookSim (bonded source, 10 CTC)	Sepolia	0x6da1A10d236607224d49De19e69885BC1f1Ad7af
RepaymentVault	Sepolia	0xD694C9f83C6D8B9Ec4DCB7A54cD3305BB050deF1
Morpho Blue singleton (Verified source)	Sepolia	0xd011EE229E7459ba1ddd22631eF7bF528d424A14
Aave v3 Pool (Verified source)	Sepolia	0x6Ae43d3271ff6888e7Fc43Fd7321a503ff738951

Repository — github.com/henessay/truthgate · Live dashboard — truthgate-six.vercel.app

Full transaction-level evidence: [docs/live-run.md](#) · Machine-readable coverage: [docs/protocol-registry.json](#) ·

Integration deep dive: [docs/attestcoin-integration-summary.md](#)

TruthGate whitepaper v1.0 · September 12, 2026 · Networks: Creditcoin CC3 Testnet (chainId 102031) × Ethereum Sepolia · Protocol: USC / Attestcoin (Gluwa)